
Esercitazione 6: Visualizzazione real-time dei dati del sensore tramite GUI MATLAB

EDOARDO NORI – 0001123033

Obiettivo: In questa esercitazione si completa la catena di ricezione del sensore radio implementando una interfaccia grafica (GUI) in MATLAB per la visualizzazione in tempo reale dei dati acquisiti tramite RTL-SDR. Partendo dai file template forniti:

- GUI_IOT_RTLSDR_template.m
- process_frame_template.m

si dovrà realizzare un sistema completo in grado di:

- acquisire i dati dal ricevitore RTL-SDR
- elaborare i pacchetti ricevuti (Es4 + Es5)
- visualizzare i dati di:
 - temperatura
 - umidità
 - pressione
 - stato batteria
- aggiornare dinamicamente i grafici in tempo reale

Parte A: Completamento della funzione `process_frame`

Rinominare il file `process_frame_template.m` in `process_frame_cognome_nome.m` e completare i blocchi mancanti. La funzione deve restituire i dati misurati da ciascun sensore in questo formato:

```
data.ID  
data.T  
data.H  
data.P  
data.B
```

Se non viene trovato un pacchetto valido: `data = []`

```
Incollare qui il codice della funzione (font Courier size 8)  
function data = process_frame_nori_edoardo(IrM, rtl_sdr_fs)  
  
% process_frame_template  
%  
% Scopo:  
% Elaborare un frame acquisito dalla SDR e restituire i dati del sensore:  
%   - ID  
%   - Temperatura  
%   - Umidità  
%   - Pressione  
%   - Batteria  
%  
% Output:  
%   data → struttura con campi:  
%       data.ID  
%       data.T  
%       data.H  
%       data.P  
%       data.B  
%  
% Se nessun pacchetto valido viene trovato:  
%   data = []
```

```

data = [];

%% === PARAMETRI DI RICEZIONE ===
%% Parametri di ricezione
thcorr = 13; % Soglia per la detection del pacchetto (idealmente 13)
rb = 2500; % bit rate in bit/s
barker13 = [1 0 1 0 1 1 0 0 1 1 1 1]; % Sequenza di Barker |-> dal TX
gx = gf( [1 1 1 0 0 1 1 1 1] ); % Pol. generatore del CRC-8
M = 5; % Fattore riduzione fs

fs = rtlsdr_fs/M; % nuova frequenza di campionamento
os = fs/rb; % fattore di sovracampionamento = campioni per simbolo
% Pesì per FIR della demodulazione non coerente:
b_dem = [ ones(1,round(os/2)), -ones(1,round(os/2)) ];
% Pesì per FIR sincronismo:
barker13_flip = fliplr( mapper( barker13, 1 ) );

% Progettazione filtro IIR di dimensioni minime, con banda passante
% centrata al tempo di bit, di larghezza 10 Hz e ripple di 1 dB.
% Frequenza di campionamento ridotta del fattore M.
Rp = 1; % Ripple in banda passante [dB]
bw = 10; % Larghezza banda passante [Hz]
% Wp deve essere un vettore contenente inizio e fine della banda passante,
% con le due frequenze normalizzate a metà della freq di campionamento.
Bal = ( rb-bw/2 ) / ( fs/2 );
Bah = ( rb+bw/2 ) / ( fs/2 );
Wp = [ Bal Bah ];
[ biir, aiir ] = cheby1( 1, Rp, Wp );

%% PER I PUNTI A-B-C-D-E usare il codice sviluppato per l'Esercitazione 5

%% === A) DEMODULAZIONE ===
% Implementare energy detector:
IrM = downsample(Ir,M);
A = filter(b_dem, 1, abs(IrM).^2);
t = 0:1/rb:size(IrM, 2)/rb - 1/rb;

%% === B) SINCRONISMO DI SIMBOLO ===
A2 = A.^2;
timew = filter( biir, aiir, A2 );
n = rtlsdr_fs / ( M * rb );
pimez = round( n/4 );
timewave = [ zeros( 1, pimez ) timew( 1:end-pimez ) ];
synchwave = sign( timewave );

%% === C) CAMPIONAMENTO ===
% Usare la funzione campionamento
[softbits,sampinst] = campionatore(A,synchwave);

%% === D) DECISIONE ===
% Convertire softbits in bit
E = max(abs(IrM))^2/(2*rb/5)*10^(rtlsdr_gain/10);
decoded = softbits > 0.8*E;

%% === E) SINCRONISMO DI PACCHETTO ===
decoded_mapped = mapper(decoded, 1);
yn = filter(barker13_flip, 1, decoded_mapped);
packsynch = yn;

%% === F) ESTRAZIONE PAYLOAD ===
T_lenD = 14;
H_lenD = 10;
P_lenD = 16;
B_lenD = 10;
datalen = T_lenD + H_lenD + P_lenD + B_lenD + 4 + 4 + 8; %lunghezza totale del
payload
for j = 1 : length(decoded)-datalen % Cerca un punto j dove la correlazione packsynch
supera la
% soglia thcorr -> inizio pacchetto rilevato.
if packsynch(j) >= thcorr %thcorr = 13
    payload = decoded(j+1:j+datalen); % è il pacchetto senza preambolo
    ID = bin2dec( num2str( payload( 1:4 ) ) );%serve per convertire i primi 4 bit
del pacchetto (payload)
    % in un numero decimale, che rappresenta l'ID del sensore
    %num2str(payload(1:4)) converte un vettore in una stringa

```

```

cur = 9;
T=[]; H=[]; P=[]; B=[];
str_T = '';
str_H = '';
str_P = '';
str_B = '';
if( payload(5) )
    T = bin2dec( num2str( payload(cur:cur+T_lenD-1) ) );
    T = (T-4000)/100;
    cur = cur + T_lenD;
    str_T = strcat( ' Temp= ', num2str(T, '%3.2f' ), '°C; ' );
end
if( payload(6) )
    H = bin2dec( num2str( payload(cur:cur+H_lenD-1) ) );
    H = H/10;
    cur = cur + H_lenD;
    str_H = strcat( ' Hum= ', num2str(H, '%3.1f' ), '%; ' );
end
if( payload(7) )
    P = bin2dec( num2str( payload(cur:cur+P_lenD-1) ) );
    P = P+50000;
    cur = cur + P_lenD;
    str_P = strcat( ' Press= ', num2str(P/100, '%5.2f' ), 'hPa; ' );
end
if( payload(8) )
    B = bin2dec( num2str( payload(cur:cur+B_lenD-1) ) );
    B = B*5/1023;
    cur = cur + B_lenD;
    str_B = strcat( ' Batt= ', num2str(B, '%2.2f' ), 'V; ' );
end
% Controllo CRC
payloadGF2 = gf( payload(1:cur+7) );
[ quoziente, remainder ] = deconv( payloadGF2, gx ); %Si calcola il resto
della divisione polinomiale in GF(2)
%con il polinomio generatore gx.
if remainder == 0
    if bin2dec( num2str( payload(5:8) ) ) == 0
        fprintf( 'Sensore %d: TEST OK\n', ID )
    else
        fprintf( 'Sensore %d: %s%s%s%s\n', ID, str_T, str_H, str_P, str_B )
    end
    %% NEW
    data.ID = ID;
    data.T = T;
    data.H = H;
    data.P = P;
    data.B = B;
    return;
else
    fprintf( 2, 'Errore ID %d\n', ID )
    fprintf( 2, 'Errore ID %d: %s%s%s%s\n', ID, str_T, str_H, str_P, str_B )
    return;
end
end
end
end

```

Parte B: Completamento della funzione GUI_IOT_RTLSDR.m

Rinominare il file GUI_IOT_RTLSDR_template.m in GUI_IOT_RTLSDR_cognome_nome.m e completare la realizzazione della GUI seguendo le istruzioni già presenti nel codice. Il file template guida passo passo nella costruzione dell'interfaccia e nell'integrazione con il sistema di ricezione.

Requisiti della GUI Seguendo le istruzioni nel template, la GUI deve contenere: 3 grafici per visualizzare la temperatura, l'umidità, la pressione, e 1 tabella: per lo stato delle batterie (ID, Volt). I grafici devono:

- aggiornarsi in tempo reale
- mostrare una curva per ogni sensore
- avere colori distinti

- avere legenda dinamica
- L'asse X deve crescere dinamicamente
- Per evitare problemi di visualizzazione dovuti ad outlier, l'asse Y deve essere fissato entro certi limiti. Ad esempio, la temperatura: [15 25] °C

La tabella deve mostrare l'ID del sensore e il livello della batteria. Anche la tabella deve essere aggiornata in tempo reale. Per verificare il corretto funzionamento della GUI:

- Avviare la GUI
- osservare l'aggiornamento dei grafici
- verificare la presenza di più sensori
- controllare la coerenza dei valori.

La GUI verrà organizzata in quattro blocchi principali:

1. Acquisizione del segnale (RTL-SDR)
2. Elaborazione del segnale (process_frame). La funzione process_frame deve essere richiamata all'interno della GUI per elaborare ogni frame acquisito dalla SDR e restituire i dati del sensore.
3. Gestione dei dati multi-sensore
4. Visualizzazione grafica e tabellare

Le principali funzioni che si consiglia di usare sono:

Funzione MATLAB	Sintassi	Descrizione
Timer MATLAB	<code>t = timer(...); start(t);</code>	Esegue automaticamente una funzione a intervalli regolari (real-time).
Callback GUI	<code>updateGUI(~,~)</code>	Funzione eseguita dal timer che aggiorna acquisizione, processing e grafici.
Struttura dati	<code>containers.Map</code>	Permette di associare ad ogni ID sensore un vettore di dati nel tempo.
Plot grafico	<code>h=plot(ax, x, y)</code>	Visualizza i dati del sensore in funzione del tempo.
Legenda dinamica	<code>legend(ax, h, labels)</code>	Mostra gli ID dei sensori attivi nel grafico.
Gestione asse X	<code>xlim(ax,[xmin xmax])</code>	Imposta i limiti dell'asse temporale.
Gestione asse Y	<code>ylim(ax,[ymin ymax])</code>	Imposta i limiti dell'asse verticale per evitare distorsioni.
Sottografici	<code>subplot(m,n,p)</code>	Divide la figura in più aree per visualizzare grafici diversi.
Pannello GUI	<code>uipanel(...)</code>	Crea un contenitore grafico con titolo (es. per la tabella batterie).
Tabella GUI	<code>uitable(...)</code>	Visualizza dati in formato tabellare (ID, tensione batteria).

Incollare qui il codice della funzione (font Courier size 8)

```
function GUI_IOT_RTLSDR_nori_edoardo()
    clc; close all;

    %% Parametri RTL-SDR e inizializzazione
    rtlcdr_id      = '0';
    rtlcdr_tunerfreq = 433.882e6;
    rtlcdr_gain     = 32.8;
    rtlcdr_fs       = 300e3;
    rtlcdr_frmlen   = 256*1024;
    rtlcdr_datatype = 'single';
    rtlcdr_ppm      = 0;
    M = 5;

    %% === RTL-SDR ===
    obj_rtlcdr = comm.SDRRTLReceiver( rtlcdr_id,...
        'CenterFrequency', rtlcdr_tunerfreq,...
        'EnableTunerAGC', false,...
        'TunerGain', rtlcdr_gain,...
        'SampleRate', rtlcdr_fs, ...
        'SamplesPerFrame', rtlcdr_frmlen,...
        'OutputDataType', rtlcdr_datatype ,...
        'FrequencyCorrection', rtlcdr_ppm );

    %% === GUI ===
    % Creare una figura MATLAB per la dashboard.
    % La figura deve:
    % - avere un titolo, es. 'Monitor Sensori IoT'
```

```

% - avere sfondo bianco
% - avere dimensioni circa [100 100 1200 700]
% - opzionale: usare la callback CloseRequestFcn per chiudere correttamente timer
e SDR
%
% Salvare la figura nella variabile:
% fig
fig = uifigure('Name', 'Monitor Sensori IoT', 'Position', [100 100 1200 700]);
fig.CloseRequestFcn = @closeApp;
fig.Theme = "light";

gl = uigridlayout(fig, [2, 2]);
gl.BackgroundColor = 'w';
gl.Padding = [20 20 20 20];
gl.RowSpacing = 20;
gl.ColumnSpacing = 20;

%% === Assi grafici ===
% La GUI deve essere composta da 4 sotto-finestre per la visualizzazione
% della temperatura, umidità, pressione e per lo stato della batteria
% A tal fine, si creino tre subplot e una tabella:
% axT -> subplot(2,2,1) per la temperatura
% axH -> subplot(2,2,2) per l'umidità
% axP -> subplot(2,2,3) per la pressione
%
% Per ciascun asse:
% - impostare il titolo, attivare la griglia
% axT = subplot(2, 2, 1, 'Parent', fig, "Color", "w");
axT = uiaxes('Parent', gl, 'BackgroundColor', 'w');
title(axT, "Temperatura", 'Color', 'k');
axT.XColor = 'k';
axT.YColor = 'k';
grid(axT, 'on');

axH = uiaxes('Parent', gl, 'BackgroundColor', 'w');
title(axH, "Umidità", 'Color', 'k');
axH.XColor = 'k';
axH.YColor = 'k';
grid(axH, 'on');

axP = uiaxes('Parent', gl, 'BackgroundColor', 'w');
title(axP, "Pressione", 'Color', 'k');
axP.XColor = 'k';
axP.YColor = 'k';
grid(axP, 'on');

%% === Zona tabella batterie ===
% Creare il quarto subplot e usarlo solo per ricavarne la posizione.
% Poi:
% 1) nascondere l'asse
% 2) creare un pannello con titolo 'Stato Batterie', usare il comando
% uipanel
% 3) creare una uitable dentro il pannello, usare il comando uitable
%
% La tabella deve avere due colonne:
% { 'ID', 'Volt' }
panel = uipanel('Parent', gl, 'Title', 'Stato Batterie', 'FontSize', 12,
'BackgroundColor', 'white', 'ForegroundColor', 'k');

gl_panel = uigridlayout(panel, [1, 1]);
gl_panel.BackgroundColor = 'w';
gl_panel.Padding = [10 10 10 10];

% uit = uitable('Parent', panel, 'ColumnName', {'ID', 'Volt'}, 'Units',
'normalized', 'Position', [0.05 0.05 0.9 0.9]);
uit = uitable('Parent', gl_panel, 'ColumnName', {'ID', 'Volt'},
'BackgroundColor', [1 1 1; 1 1 1]); % Righe tutte bianche

%% === STORAGE DATI ===
% Creare quattro containers.Map per memorizzare i dati nel tempo:
% T_data -> temperatura
% H_data -> umidità
% P_data -> pressione
% B_data -> batteria
%
% Usare come input:
% 'KeyType','int32'
% 'ValueType','any'

```

```

T_data = containers.Map('KeyType', 'int32', 'ValueType', 'any');
H_data = containers.Map('KeyType', 'int32', 'ValueType', 'any');
P_data = containers.Map('KeyType', 'int32', 'ValueType', 'any');
B_data = containers.Map('KeyType', 'int32', 'ValueType', 'any');

%% === Colori sensori ===
% Definire una matrice colors per i colori dei sensori, da poter richiamare
% nei plot
colors = ['k', 'r', 'g', 'b', 'y', 'c'];

%% === TIMER REAL-TIME ===
% Creare un timer MATLAB che:
% - usi ExecutionMode = 'fixedSpacing'
% - abbia Period = 1
% - usi BusyMode = 'drop'
% - richiami la funzione updateGUI
%
% Salvare il timer nella variabile:
%   t
%
% Poi avviare il timer con start(t)
t = timer('ExecutionMode', 'fixedSpacing', 'Period', 1, 'BusyMode', 'drop',
'TimerFcn', @updateGUI);
start(t);

%% === CALLBACK UPDATE ===
function updateGUI(~, ~)
    %% === ACQUISIZIONE ===
    % 1) acquisire un frame dalla SDR con step(...)
    % 2) trasporre il vettore ricevuto
    % 3) fare downsample del segnale con fattore M
    %
    % Variabili richieste:
    %   Ir
    %   IrM

    Ir = step(obj_rtlsdr)';
    % struttura = load("samplesIQpacket.mat");
    % Ir = struttura.Ir;

    IrM = downsample(Ir, M);

    %% === PROCESSING ===
    % Chiamare la funzione process_frame per elaborare il frame acquisito.
    % La funzione deve restituire una struct data con campi:
    %   data.ID
    %   data.T
    %   data.H
    %   data.P
    %   data.B
    %
    % Suggerimento:
    %   data = process_frame(IrM, rtlsdr_fs);

    data = process_frame_nori_edoardo(IrM, rtlsdr_fs);

    %% === CONTROLLO DATI ===
    % Se data e' vuoto, uscire dalla callback senza aggiornare la GUI.

    if isempty(data) || ~isfield(data, 'ID')
        return;
    end

    % Estrarre l'ID del sensore nella variabile:
    %   ID
    ID = int32(data.ID);

    %% === SALVATAGGIO TEMPORANEO ===
    % Aggiornare le quattro mappe T_data, H_data, P_data, B_data.
    %
    % Per T, H e P:
    % - se la chiave ID non esiste ancora, inizializzarla col primo valore
    % - altrimenti concatenare il nuovo dato al vettore esistente
    %
    % Per B:
    % - salvare semplicemente l'ultimo valore ricevuto
    %
    % Usare:

```

```

% isempty(...) --> per verificare se era già stato letto un
% valore di quel tipo
% isKey(...) --> per verificare se era già stato letto un
% valore per quel sensore
%
% Completare qui sotto i blocchi per:
% - temperatura
% - umidità
% - pressione
% - batteria

if ~isKey(T_data, ID) || isempty(T_data(ID))
    T_data(ID) = double(data.T);
else
    T_data(ID) = [T_data(ID), double(data.T)];
end

if ~isKey(H_data, ID) || isempty(H_data(ID))
    H_data(ID) = double(data.H);
else
    H_data(ID) = [H_data(ID), double(data.H)];
end

if ~isKey(P_data, ID) || isempty(P_data(ID))
    P_data(ID) = double(data.P);
else
    P_data(ID) = [P_data(ID), double(data.P)];
end

B_data(ID) = double(data.B);

%% === PARAMETRO ASSE X ===
% Impostare STEP = X sec, esempio 5.
% Questo valore servirà per far crescere l'asse X a blocchi di X s.
STEP = 5;

% Plotting
% Si è usata una funzione per ridurre l'overhead dovuto alla
% ricopiatura delle stesse istruzioni per axT, axH, axP.
updatePlot(axT, T_data, 'Temperatura (°C)', [10 35], STEP);
updatePlot(axH, H_data, 'Umidità (%)', [10 90], STEP);
updatePlot(axP, P_data, 'Pressione (Pa)', [100600 102000], STEP);

%% === TABELLA BATTERIE ===
% Creare una cell array chiamata batt contenente due colonne:
% {ID, Volt}
%
% Scorrere le chiavi di B_data e aggiornare la tabella:
% set(uit, 'Data', batt);
ids = keys(B_data);
batt_cell = cell(length(ids), 2);
for i = 1:length(ids)
    curr_id = ids{i};
    batt_cell{i, 1} = double(curr_id);
    batt_cell{i, 2} = double(B_data(curr_id));
end
uit.Data = batt_cell;
end

function updatePlot(ax, map, labY, y_lim, STEP)
% Aggiorna il grafico passato come ax con il valore in map
% corrispondente alla chiave. Imposta la label e il limite su y,
% poichè per x è uguale per i 3 grafici. STEP viene passato uguale
% per tutti come incremento del tempo (asse x).
cla(ax);
hold(ax, 'on');
ks = keys(map);
labels = {};
maxL = 0;

for i = 1:length(ks)
    k = ks{i};
    valori = map(k);
    maxL = max(maxL, length(valori));

    c_idx = mod(i-1, length(colors)) + 1;

    x_vals = 1:length(valori);

```

```

        plot(ax, x_vals, double(valori), 'Color', colors(c_idx), 'LineWidth',
1.5);

        labels{end+1} = sprintf('ID %d', double(k));
    end

    ylabel(ax, labY); xlabel(ax, 'Tempo [s]');
    ylim(ax, y_lim);

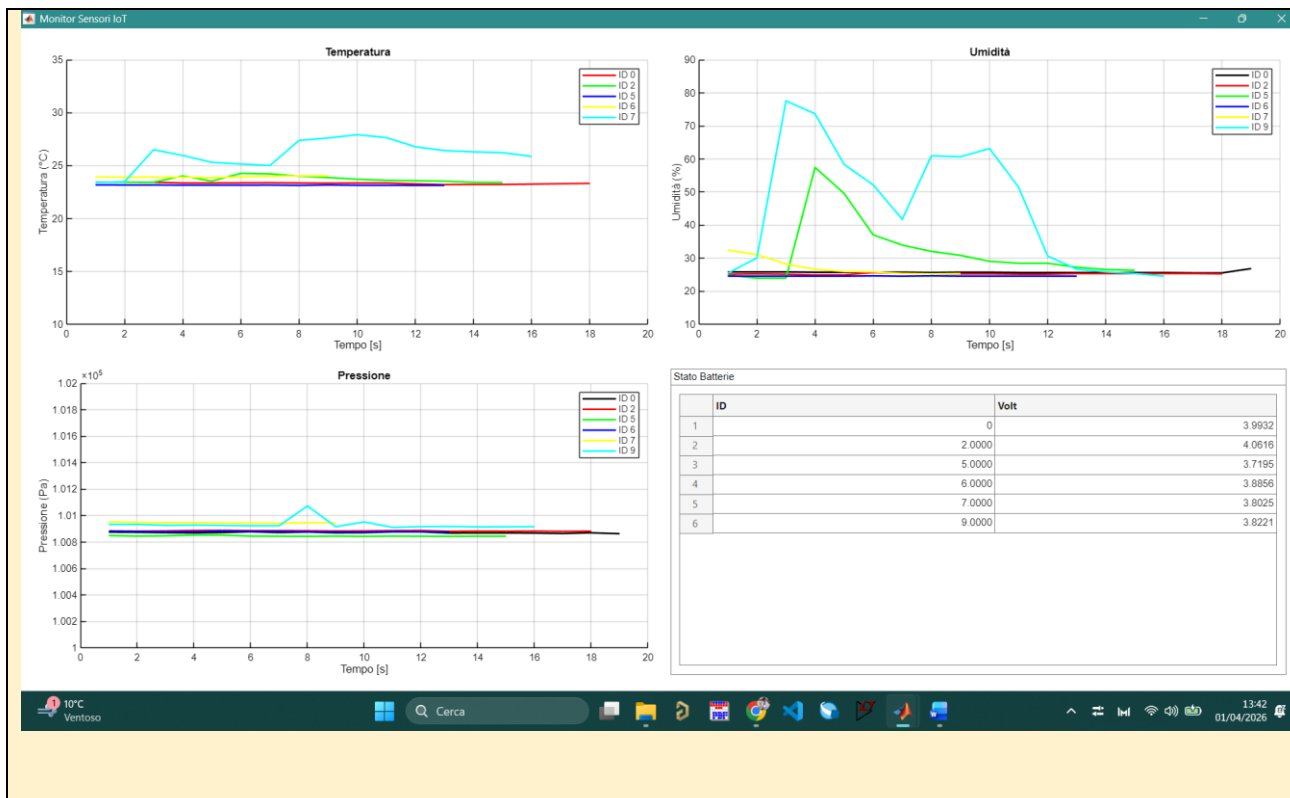
    if maxL > 0
        xlim(ax, [0, ceil(maxL/STEP)*STEP]);
    end

    if ~isempty(labels)
        legend(ax, labels, 'Location', 'northeast');
    end
    grid(ax, 'on');
end

%% === CHIUSURA APPLICAZIONE ===
function closeApp(~, ~)
    % Completare questa funzione in modo che:
    % 1) fermi il timer
    % 2) cancelli il timer
    % 3) rilasci la SDR
    % 4) chiuda la figura
    %
    % Usare blocchi try/catch per evitare errori in chiusura.
    %
    % Suggerimento:
    %   stop(t)
    %   delete(t)
    %   release(obj_rtlsdr)
    %   delete(gcf)
    fprintf('Chiusura in corso...\n');
    try stop(t); catch; end
    try delete(t); catch; end
    try release(obj_rtlsdr); catch; end
    try delete(fig); catch; end
end
end

```

Incollare qui uno screenshot della GUI.



Parte C: Estensioni (Facoltative)

- Aggiungere pulsante Start/Stop acquisizione
- Aggiungere pulsante per Reset dei grafici
- Mostrare/filtrare solo alcuni sensori selezionati
- Ordinare la tabella per livello di batteria
- Implementare un sistema che identifichi valori anomali (outlier) e li scarti senza mostrarli sui grafici
- Aggiungere alla GUI un pannello con le statistiche per sensore (media, varianza, valore min/max)

Istruzioni:

1. Grafici e risultati prodotti devono recare sempre le relative unità di misura (ms, Hz).
2. Fare le opportune sostituzioni nelle parti evidenziate in verde prima della consegna.
3. Rinominare questo file sostituendo nome e cognome.
4. Caricare su Virtuale entro la scadenza comunicata a lezione:
 - questa relazione compilata in ogni sua parte in **FORMATO PDF**
 - tutto il codice sviluppato (script, funzioni) e i file generati (**.zip o .rar**)
 - il file GUI_IOT_RTLSDR_cognome_nome.m
 - il file process_frame_cognome_nome.m

Attenzione: non usare **MAI** la funzione “screenshot” del vostro SO. Salvare **SEMPRE** l'immagine con il comando `print()` posizionato dopo il `plot()` con una risoluzione di **almeno 300dpi** (uno qualunque di questi formati va bene):

```
print(gcf, 'nomefile.png', '-dpng', '-r300');
print(gcf, 'nomefile.jpg', '-djpeg', '-r300');
print(gcf, 'nomefile.pdf', '-dpdf', '-r300');
```

oppure, ancora meglio, in formato vettoriale come l'Encapsulated PostScript (EPS):

```
print(gcf, 'nomefile.eps', '-deps');
```

Good practice

Comment your code; describe each module with the following information:

- Name and purpose of the Module
- Description of the Module (in brief)

- *Example of usage: input and output*
- *Author, date and version*

Example in Matlab:

```
function [mymod] = mymodexp(b,e,n)

    % mymodexp(b,e,n) gives  $b^e \bmod n$ , where b, e, n are integers
    %
    % iterative implementation based on the property
    %  $(a \cdot (a^m \bmod n)) \bmod n = a^{(m+1)} \bmod n$  to avoid numerical overflow
    %
    % example of usage: mymodexp(137,56,14731) gives as output 6849
    %
    % Andrea Giorgetti, Oct. 5, 2020, version 1.2 %
```
